



Proiect PCLP3

Mihai Nan

Andrei-Daniel Voicu, George-Alexandru Tudor

Departamentul de Calculatoare • Data postării: 4 mai 2026

Deadline: 27.05.2026, ora 23:59

Notă

Proiectul poate fi realizat **individual** sau **în echipă de 2 studenți**. În cazul în care alegeți să realizați proiectul individual, veți realiza doar prima parte. Dacă alegeți să realizați proiectul în echipă, fiecare student își va alege o parte.

Cuprins

1	Introducere	1
2	Partea I – Construirea și explorarea unui dataset tabelar	1
2.1	Cerințe detaliate	1
3	Partea a II-a – Prelucrarea avansată a datelor și compararea modelelor	2
3.1	Cerințe detaliate	2
4	Bonus	3
4.1	Opțiunea 1 – Interfață grafică	3
4.2	Opțiunea 2 – Propunerea unei probleme pentru platforma Olimpiadei de Inteligență Artificială	4
5	Punctaj	6

1 Introducere

Python a devenit indispensabil în domeniul *Data Science* și *Inteligență Artificială*, mai ales în explorarea datelor și dezvoltarea de modele, datorită ecosistemului său bogat de biblioteci specializate. Cu biblioteci precum **numpy**, **pandas**, **matplotlib** și **seaborn**, Python oferă instrumente puternice pentru manipularea, vizualizarea și analiza datelor statistice. Acestea permit cercetătorilor și analiștilor să exploreze seturile de date, să identifice modele și tendințe și să obțină înțelegeri profunde din datele brute.

2 Partea I – Construirea și explorarea unui dataset tabelar

În prima parte a proiectului, veți construi un set de date tabelar propriu, folosind una dintre următoarele metode:

1. **Web scraping / API-uri publice** – extragerea de informații de pe internet (ex: OpenWeatherMap, SpaceX Launch Data);
2. **Generare sintetică** – construirea unui dataset cu sens contextual (ex: simularea datelor medicale despre pacienți, date financiare despre companii, date despre specii de plante/animale);
3. **Extindere sau combinare de dataset-uri existente** – pornirea de la unul sau mai multe dataset-uri existente și realizarea unor modificări relevante: adăugarea de coloane calculate, combinarea mai multor surse, generarea de etichete, simularea de valori lipsă, introducerea de zgomot, agregări sau transformări semnificative.

Atenție!

Nu este permisă folosirea unui dataset existent fără modificări relevante. Setul de date final trebuie să fie rezultatul unei contribuții proprii semnificative: fie generați un dataset sintetic cu sens, fie colectați date, fie modificați relevant un dataset existent, fie combinați două sau mai multe seturi de date existente.

2.1 Cerințe detaliate

1. **Tipul problemei.** Setul de date propus trebuie destinat fie unei probleme de regresie, fie uneia de clasificare. Tipul problemei trebuie specificat clar în documentație.
2. **Structura setului de date.** Setul final se împarte în două subseturi:
 - *Subset de antrenare* – cel puțin **500** de instanțe;
 - *Subset de testare* – cel puțin **200** de instanțe.Împărțirea se realizează prin extragere randomizată (ex: **scikit-learn**, **pandas**).
3. **Numărul minim de caracteristici.** Fiecare instanță trebuie să aibă minimum **8 coloane relevante**, inclusiv coloana țintă. Dintre acestea se vor folosi cel puțin **3 tipuri diferite de date** (numere întregi, numere reale, valori categoriale, șiruri de caractere etc.).
4. **Salvarea dataset-urilor.** Exportul subsetului de antrenare și al subsetului de testare în fișiere CSV separate.
5. **Documentare.** Explicarea modului de construcție a setului de date: surse, metode de generare, ipoteze și procesări suplimentare. Se va evidenția clar contribuția proprie asupra datelor.
6. **Analiza exploratorie a datelor (EDA complex).** Se va realiza o explorare detaliată pentru fiecare dintre cele două subseturi (antrenare și testare), care să includă obligatoriu:
 - a) *Analiza valorilor lipsă* – număr și procent pe coloană; strategii de tratare (imputare, ștergere).

- b) *Statistici descriptive* – utilizarea `describe()` și interpretarea statisticilor pentru variabile numerice și categorice.
 - c) *Analiza distribuției variabilelor* – histogramă pentru fiecare caracteristică numerică; `countplot`/`barplot` pentru variabilele categorice.
 - d) *Detectarea outlierilor* – `boxplot`-uri sau alte tehnici (regula IQR).
 - e) *Analiza corelațiilor* – matrice de corelații (heatmap) pentru variabilele numerice.
 - f) *Analiza relațiilor cu variabila țintă* – scatter plots sau violin plots, în funcție de tipul problemei.
 - g) *Comentarii și interpretări personale*. Fiecare grafic trebuie însoțit de o scurtă interpretare care să răspundă la: *Ce observăm? Ce suspiciuni/idei putem formula? Ce preprocesări ar trebui să aplicăm?*
7. **Antrenarea și evaluarea unui model de bază.** Se va antrena un model simplu din biblioteca `scikit-learn`, potrivit pentru problema aleasă (ex: regresie liniară, logistic regression, random forest).
- Modelul se antrenează pe subsetul de antrenare și se evaluează pe subsetul de testare.
 - Se raportează și se interpretează rezultatele utilizând metrice adecvate:
 - *Regresie*: RMSE, MAE sau R^2 ;
 - *Clasificare*: acuratețe, precizie, recall, F1-score.
 - Se includ grafice relevante pentru performanță (matrice de confuzie, grafice de erori etc.).

3 Partea a II-a – Prelucrarea avansată a datelor și compararea modelelor

În această parte a proiectului, se va continua lucrul pornind de la datasetul realizat de colegul de echipă la [Partea I](#).

3.1 Cerințe detaliate

1. **Prelucrarea datelor.** Se vor aplica tehnici de prelucrare a datelor, după caz:
 - Normalizare sau standardizare pentru variabilele numerice;
 - Encodare pentru variabilele categorice (`OneHotEncoder`, `LabelEncoder`);
 - Înlocuirea valorilor lipsă prin metode adecvate (medie, mediană, modă, imputare avansată).Se va explica pentru fiecare tehnică *de ce* a fost necesară și *ce impact* are asupra modelelor de machine learning (comparativ: rularea modelului cu / fără aplicarea tehnicii).
2. **Analiza exploratorie a datelor (EDA complex) după aplicarea prelucrărilor.** Se va realiza o explorare detaliată pentru fiecare dintre cele două subseturi (antrenare și testare), cu aceleași puncte ca la Partea I:
 - a) Analiza valorilor lipsă (număr și procent pe coloană; strategii de tratare);
 - b) Statistici descriptive (`describe()` și interpretarea lor);
 - c) Analiza distribuției variabilelor (histograme + `countplot`/`barplot`);
 - d) Detectarea outlierilor (`boxplot` / IQR);
 - e) Analiza corelațiilor (heatmap);
 - f) Analiza relațiilor cu variabila țintă (scatter / violin plots);
 - g) Comentarii și interpretări personale (*Ce observăm? Ce suspiciuni/idei? Ce preprocesări?*).Puteți colabora cu colegul de echipă și să preluați codul realizat de el, pe care să-l adaptați pentru a fi aplicat pe seturile de date procesate. Identificați și documentați eventualele *limitări*

sau *puncte tari* ale datasetului produs de colegul de echipă.

3. **Antrenarea și compararea a cel puțin 3 algoritmi diferiți.** Se vor alege minimum 3 modele diferite din `scikit-learn`, potrivite tipului de problemă:

- *Regresie*: Linear Regression, Ridge Regression, Decision Tree Regressor, Random Forest Regressor, SVR etc.;
- *Clasificare*: Logistic Regression, Decision Tree Classifier, Random Forest Classifier, SVM, KNN etc.

Fiecare model va fi antrenat pe datele prelucrate (subsetul de antrenare) și evaluat pe subsetul de testare.

4. **Evaluarea performanței.** Modelele se compară utilizând **aceeași metrică** (aleasă de voi) pentru toate:

- *Regresie*: RMSE, MAE, R^2 etc.;
- *Clasificare*: acuratețe, F1-score, ROC AUC etc.

Se va construi un **tabel comparativ** care să includă numele algoritmului și valorile obținute pentru fiecare metrică relevantă. Pentru o analiză mai detaliată:

- *Clasificare* – matrice de confuzie pentru fiecare model și, dacă e relevant, curbe ROC;
- *Regresie* – scatter plots (valoare reală vs. valoare prezisă) și/sau distribuția reziduurilor.

Aceste vizualizări trebuie să ajute la interpretarea performanței și identificarea punctelor slabe ale fiecărui model.

4 Bonus

Indiferent de ce parte alegeți să rezolvați, puteți obține un bonus de **20 de puncte** prin realizarea uneia dintre următoarele două activități. *Se alege o singură variantă.*

4.1 Opțiunea 1 – Interfață grafică

Se poate dezvolta o interfață grafică care să permită utilizatorului să introducă valori pentru variabilele de intrare și să vizualizeze predicțiile și performanțele modelelor de învățare automată.

Input pentru date

- Câmpuri de introducere a valorilor pentru variabilele numerice și categorice (ex: text pentru valori numerice, dropdown pentru variabilele categorice);
- Un buton *Predicție* care preia datele introduse și le pasează unui model antrenat.

Predicție și vizualizare

- La apăsarea butonului, aplicația rulează modelele selectate și afișează predicțiile pentru fiecare algoritm;
- *Clasificare* – se pot afișa probabilitățile fiecărei clase;
- *Regresie* – se afișează valoarea prezisă și, dacă există, o comparație cu valoarea reală.

Vizualizări grafice

- *Clasificare* – matrice de confuzie (și, dacă e relevant, curbe ROC) pentru fiecare model;

- *Regresie* – scatter plot cu valori reale vs. prezise și/sau distribuția reziduurilor pentru fiecare model.

4.2 Opțiunea 2 – Propunerea unei probleme pentru platforma Olimpiadei de Inteligență Artificială

Pornind de la setul de date construit la [Partea I](#), veți formula și propune o problemă care să poată fi publicată pe platforma Olimpiadei de Inteligență Artificială¹.

Problema trebuie să valorifice datasetul realizat în proiect și să fie potrivită pentru elevi de liceu. Scopul este ca datele construite la Partea I să devină suportul unei probleme aplicate, cu enunț clar, date de intrare și ieșire bine definite, evaluator automat și o soluție de referință.

Cerință de originalitate

Problema propusă trebuie să fie **originală** și să pornească de la datasetul realizat în cadrul proiectului. Nu este acceptată reformularea unei probleme deja publicate.

Structura arhivei pentru bonus

Propunerea se va încărca sub forma unei arhive care conține obligatoriu următoarele fișiere:

Conținutul arhivei pentru problema propusă

<code>statement.md</code>	enunțul problemei, scris în format Markdown
<code>train.csv</code>	datele de antrenare, cu valorile țintă incluse
<code>test.csv</code>	datele de testare, fără valorile țintă
<code>ground_truth.csv</code>	răspunsurile corecte pentru datele de testare și/sau pentru subtask-uri
<code>evaluation.py</code>	evaluatorul automat al unui fișier de submisie
<code>solution.py</code>	soluția de referință, recomandată, care obține punctaj maxim

1. `statement.md` Fișierul `statement.md` conține enunțul complet al problemei în format Markdown. Nu se vor folosi fișiere `.docx` sau PDF pentru enunț.

Enunțul trebuie să includă:

- titlul problemei;
- povestea sau contextul problemei;
- descrierea datelor din `train.csv` și `test.csv`;
- explicația fiecărei coloane;
- cerințele problemei;
- descrierea formatului submisiei;
- exemple de submisie;
- descrierea metricii de evaluare;
- eventuale subtask-uri, cu punctajele aferente.

¹<https://platform.olimpiada-ai.ro/en>

Atenție!

Enunțul trebuie să fie suficient de clar încât un participant să poată înțelege ce trebuie să prezică sau să calculeze fără să aibă acces la `ground_truth.csv`.

2. `train.csv` Fișierul `train.csv` conține datele de antrenare. Acesta trebuie să includă:

- un identificator unic pentru fiecare instanță, de exemplu `id`;
- coloanele explicative;
- coloana țintă, adică valoarea care trebuie prezisă sau determinată.

3. `test.csv` Fișierul `test.csv` conține datele pe care se va face evaluarea. Acesta trebuie să aibă aceeași structură ca `train.csv`, dar **fără coloana țintă**.

Atenție!

`test.csv` nu trebuie să conțină răspunsurile corecte. Răspunsurile corecte vor fi păstrate separat în `ground_truth.csv`.

4. `ground_truth.csv` Fișierul `ground_truth.csv` conține răspunsurile corecte folosite de evaluatorul automat. Formatul standard este:

```
id,answer
1,0
2,1
3,0
4,1
...
```

Pentru probleme cu mai multe subtask-uri, se poate folosi formatul:

```
subtaskID,datapointID,answer
1,1,127
2,1,5.1
3,227846,1
3,227847,0
...
```

5. `evaluation.py` Fișierul `evaluation.py` trebuie să implementeze evaluarea automată a unui fișier de submisie. Evaluatorul va primi:

- fișierul de submisie al participantului;
- fișierul `ground_truth.csv`;
- eventual alte fișiere necesare evaluării.

Evaluatorul trebuie să:

- verifice dacă formatul submisiei este corect;
- compare răspunsurile participantului cu răspunsurile corecte;
- calculeze scorul final folosind metrica aleasă;
- afișeze scorul obținut.

Metricile pot fi alese în funcție de tipul problemei:

- *Clasificare*: accuracy, F1-score, F1 macro, ROC AUC;
- *Regresie*: MAE, RMSE, R^2 ;
- *Răspunsuri exacte*: equality score.

6. solution.py Fișierul `solution.py` conține o soluție completă de referință. Scriptul trebuie să ruleze end-to-end pe fișierele `train.csv` și `test.csv` și să producă un fișier de submit valid.

Soluția trebuie să obțină punctaj maxim sau un punctaj justificat prin baremul propus.

Cerințe suplimentare

Pe lângă fișierele de mai sus, arhiva va conține și un scurt document, în format PDF sau Markdown, care să includă:

- **Justificarea alegerii problemei** – de ce datasetul construit se potrivește problemei propuse;
- **Competențele evaluate** – de exemplu: manipularea datelor tabelare, filtrări, agregări, statistici, predicții simple, clasificare sau regresie;
- **Estimarea dificultății** – ușor, mediu sau dificil, cu justificare;
- **Baremul de evaluare** – punctajul pe subtask-uri și metrica folosită;
- **Descrierea soluției de referință** – pașii principali prin care `solution.py` obține rezultatele.

5 Punctaj

Proiectul va fi încărcat pe Moodle de fiecare membru al echipei (fiecare student își încarcă partea lui), sub forma unei arhive **.zip** cu următorul conținut.

Partea I

Structura arhivei ParteaI/	
Surse/	toate fișierele cu cod folosite (.py / .ipynb)
README.pdf	histograme, grafice, documentația completă
train.csv, test.csv	cele două subseturi ale setului de date realizat

În fișierul `README` veți descrie modul de rezolvare pentru fiecare cerință din Partea I, răspunsurile la întrebările din cerință și alte observații. **Prima linie** a fișierului va conține numele complet, seria și grupa studentului care a rezolvat Partea I.

Partea a II-a

Structura arhivei ParteaII/	
Surse/	toate fișierele cu cod folosite (.py / .ipynb)
Date/	toate fișierele rezultate din modificări / prelucrări ale setului de date
README.pdf	graficele create și documentația completă

În fișierul **README** veți descrie modul de rezolvare pentru fiecare cerință din Partea a II-a, răspunsurile la întrebările din cerință și alte observații. **Prima linie** a fișierului va conține numele complet, seria și grupa studentului care a rezolvat Partea a II-a.

Distribuția punctajului

Partea I	
Cerințe (40% cod + 40% rezultat + 20% documentație)	50 p
Bonus – Interfață grafică <i>sau</i> propunere problemă pentru platforma OIA	20 p
Total maxim acordat	70 p
Partea a II-a	
Cerințe (40% cod + 40% rezultat + 20% documentație)	50 p
Bonus – Interfață grafică <i>sau</i> propunere problemă pentru platforma OIA	20 p
Total maxim acordat	70 p

Atenție!

- Pentru a primi punctaj, trebuie să **prezentați** proiectul în ultima săptămână a semestrului.
- Toate soluțiile trimise vor fi verificate folosind o unealtă pentru **detectarea plagiatului**. În cazul depistării codului copiat (de pe Internet, de la colegi, din surse generate cu tool-uri tip ChatGPT), **întregul punctaj pentru proiect este anulat**.
- Pentru orice întrebare puteți folosi forumul.
- Pentru bonus alegeți **o singură** variantă: realizarea unei interfețe grafice (4.1) sau propunerea unei probleme pentru platforma Olimpiadei de Inteligență Artificială (4.2).